WEEK 6 · GIT

Git & GitHub — 기초

버전 관리 시스템(VCS)의 이해와 Git 기본 워크플로우

목차

01 VCS (Version Control System)란?

02 Git 소개

03 Git 설치와 기본 설정

04 Git 기본 워크플로우 (add · commit · log)

05 .gitignore로 특정 파일 제외하기

06 실습: 나만의 첫 Git 저장소 만들기

1.1

VCS (Version Control System)란?

프로그램 코드의 모든 변경 이력을 저장소(Repository)에 기록하고 관리하는 시스템입니다

- 파일이 바뀔 때마다의 기록을 "커밋(commit)"이라는 단위로 로그에 남깁니다.
- 덕분에 원하는 시점의 파일 상태로 언제든 되돌아갈 수 있습니다.
- 브랜치(branch)를 만들면 기존 코드에 영향을 주지 않고 새로운 기능을 독립적으로 개발할 수 있습니다.



핵심 각 점이 하나의 커밋(commit)입니다.
브랜치는 독립적인 작업 흐름을 만들고, 필요할 때 다시 main으로 병합될(merge) 수 있습니다.

1.1

VCS 없이 작업한다면?

버전 관리 없이 파일을 관리하면 이렇게 되기 쉽습니다.

VCS 없이 (파일 여러 개)

```
report_final.docx
```

```
report_final_v2.docx
```

```
report_final_REALLY_final.docx
```

어떤 파일이 최신인지, 무엇이 왜 바뀌었는지 알기 어렵습니다.

Git 사용 시 (파일 1개 + 커밋 이력)

report.docx (최신 버전, v3)

버전	커밋 메시지 (변경 요약)
v3	"결론 챕터 보강" + 결론 문단 2개 추가
v2	"표 3개 추가" + 비교표, 매출표, 일정표 삽입
v1	"초안 작성" + 전체 내용 최초 작성

파일은 단 1개만 유지하고, 커밋 메시지와 버전별 스냅샷(전체 내용)은 Git 이 별도로 기록합니다.

1.1

왜 VCS를 사용하는가?

1

백업

코드 변경 실수나 디스크 손상에 대비한 백업 역할을 합니다.

2

이력 관리

모든 과거 버전을 아카이브로 보관 — 필요하면 특정 시점으로 되돌릴 수 있습니다.

3

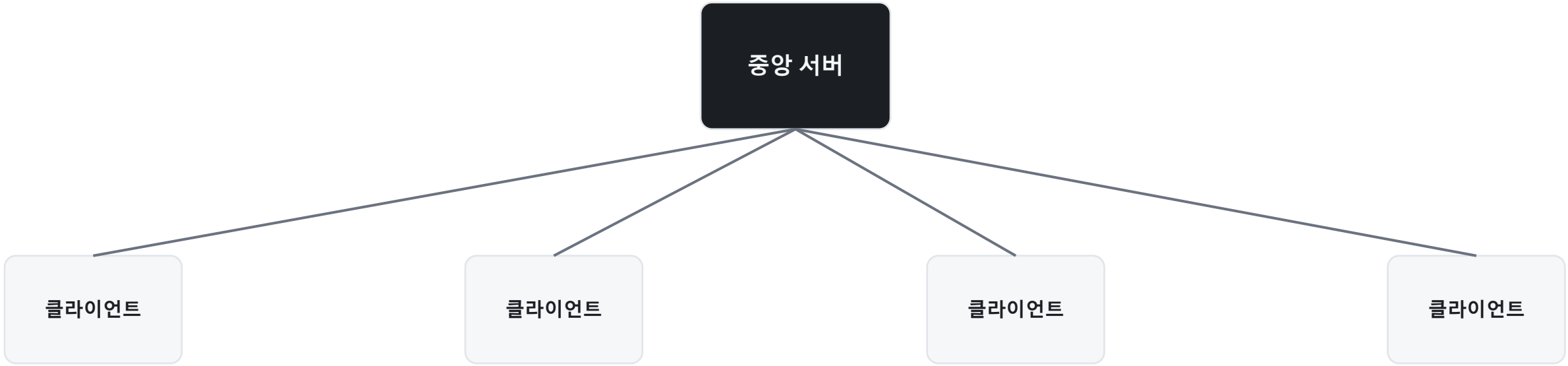
협업

여러 사람이 동시에 작업해도 변경 사항을 안전하게 통합할 수 있는 프로젝트 관리 도구입니다.

1.2

중앙집중형 VCS (Centralized)

예: CVS, Subversion(SVN)

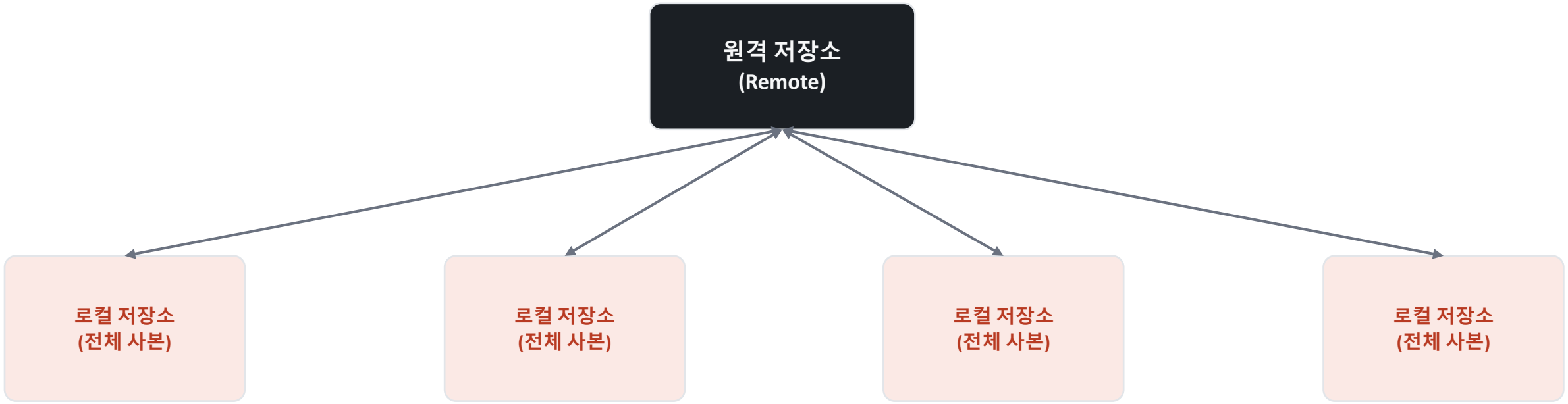


특징	설명
구조	하나의 중앙 서버가 모든 버전 정보를 보관
동작	클라이언트는 필요할 때 서버에서 파일을 내려받음(checkout)
위험	서버가 다운되면 아무도 협업을 진행할 수 없음

1.2

분산형 VCS (Distributed)

예: Git, Mercurial — 오늘날 사실상 표준



핵심 각 사용자가 전체 버전 이력이 담긴 저장소를 통째로 복사해 갖고 있습니다.
원격 서버가 잠시 멈춰도 로컬에서 커밋·조회 등 대부분의 작업을 계속할 수 있고, 이후 원격 저장소와 다시 동기화해 자신의 변경 사항을 공유할 수 있습니다.

2.1

Git 소개

탄생	2005년, 리누스 토르발스(Linus Torvalds)가 리눅스 커널 개발을 지원하기 위해 설계
분류	분산 버전 관리 시스템 (Distributed VCS)
공식 사이트	https://git-scm.com
현재	GitHub, GitLab 등 대부분의 코드 호스팅 플랫폼이 Git 기반



2.1

Git의 저장 방식: Snapshot

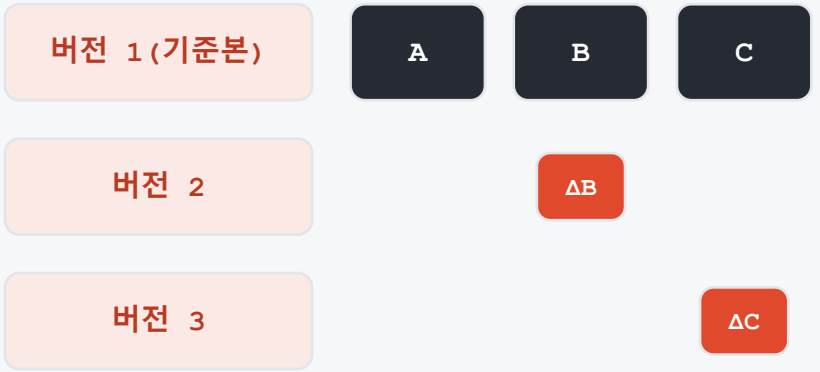
Git은 변경 사항이 아니라, 매 커밋마다 전체 상태의 "스냅샷"을 저장합니다.

Git — Snapshot 방식



그 커밋에서 바뀐 파일만 새로 저장되고, 나머지는 이전 스냅샷에 대한 링크만 남깁니다.

CVS · Subversion — Delta 방식



- 파일을 "기준본 + 변경분(delta)"의 연속으로 취급합니다.
- 각 버전을 보려면 델타를 순서대로 적용(재구성)해야 합니다.
- 병합(merge) 작업이 상대적으로 느릴 수 있습니다.

3.1

Git 설치하기

공식 사이트: https://git-scm.com

설치 및 버전 확인

```
# Amazon Linux 2023 / RHEL 계열
$ sudo dnf install -y git

# Ubuntu / Debian 계열
$ sudo apt-get install -y git

# macOS (Homebrew)
$ brew install git

$ git --version
git version 2.43.0
```

TIP Windows 사용자는 git-scm.com 설치 프로그램에 포함된 "Git Bash"를 사용하거나, WSL2(Windows Subsystem for Linux)를 설치해 리눅스와 동일한 환경에서 실습할 수 있습니다.

3.2

Git 기본 설정 (git config)

커밋을 남기기 전에, 내 이름과 이메일을 먼저 등록합니다.

git config

```
$ git config --global user.name "Hong Gildong"
$ git config --global user.email "gildong@example.com"

$ git config --global init.defaultBranch main
# 새 저장소의 기본 브랜치 이름을 main으로 지정 (최신 관례)

$ git config --list
# 현재 설정된 값 전체 확인
```

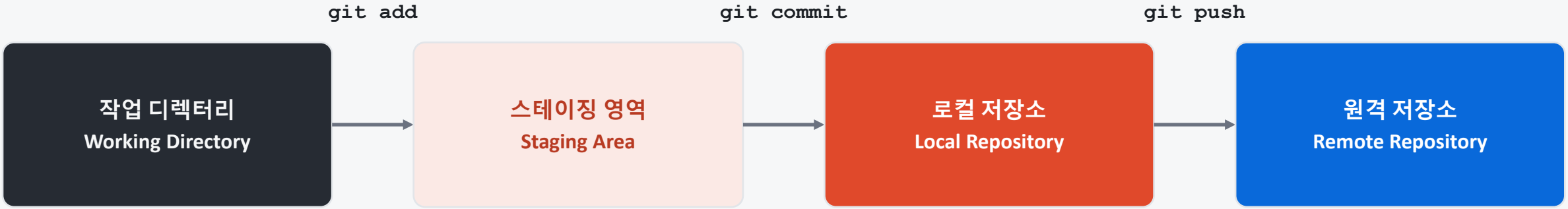
안내 이 설정은 <사용자 홈 디렉터리>/.gitconfig 파일에 저장됩니다.
--global 없이 특정 저장소에서만 다르게 설정하고 싶다면 그 저장소 폴더 안에서 --global 없이 실행합니다.

최신 변경 GitHub은 2020년부터 기본 브랜치 이름을 master 대신 main으로 사용합니다. 이 강의에서도 main을 기준으로 설명합니다.

4.1

Git 기본 워크플로우

파일은 4단계 공간을 거치며 관리됩니다 — 이번 시간에는 앞의 3단계를 다룹니다.



다음 시간(원격 저장소 편)에서 다룸

스테이징 영역이란? 커밋에 담을 변경 사항을 미리 골라 담아두는 임시 공간입니다. 작업 디렉토리의 변경을 한꺼번에 커밋하지 않고 원하는 파일만 골라 담을 수 있어, 더 정확하고 의미 있는 단위로 커밋을 나눌 수 있게 해줍니다.

이번 시간 범위 "파일을 고치고(작업 디렉터리) → 담아서(스테이징) → 저장한다(로컬 저장소)"는 흐름을 반복해서 익힙니다.
원격 저장소로 보내는 git push는 다음 주 "Git RemoteRepo"에서 다룹니다.

4.2

Git으로 관리되는 프로젝트 시작하기

Git 프로젝트를 시작하는 방법은 크게 두 가지입니다.

1

① 새 프로젝트 시작

빈 디렉터리에서 git init으로 새 저장소를 만듭니다.

```
$ mkdir hello-git && cd hello-git
$ git init
```

2

② 기존 프로젝트 복제

GitHub·Bitbucket 등 GIT 호스트에서 이미 있는 프로젝트를 내려받습니다.

```
$ git clone <remote-url>
```

이번 시간 ①번 방법(새 프로젝트 시작)을 기준으로 실습합니다. ②번 clone은 다음 주 원격 저장소 편에서 자세히 다룹니다.

4.2

① 작업 디렉터리 준비하기

hello-git 프로젝트에 파일 2개를 준비합니다.

Hello.java

```
// Hello.java
public class Hello {
  public static void main(String[] args) {
    System.out.println(
      "Hello, world from GIT!");
  }
}
```

README.md

```
# README.md
This is a README file for
the Hello-world project.
```

안내 README.md는 프로젝트를 소개하는 문서로, 관례적으로 저장소 최상위에 둡니다. GitHub에서는 저장소 첫 화면에 자동으로 표시됩니다.

4.2

② git init — 빈 저장소 만들기

프로젝트 루트 디렉터리에서 git init을 실행합니다.

git init

```
$ cd hello-git
$ git init
Initialized empty Git repository in /home/ec2-user/hello-git/.git/
```

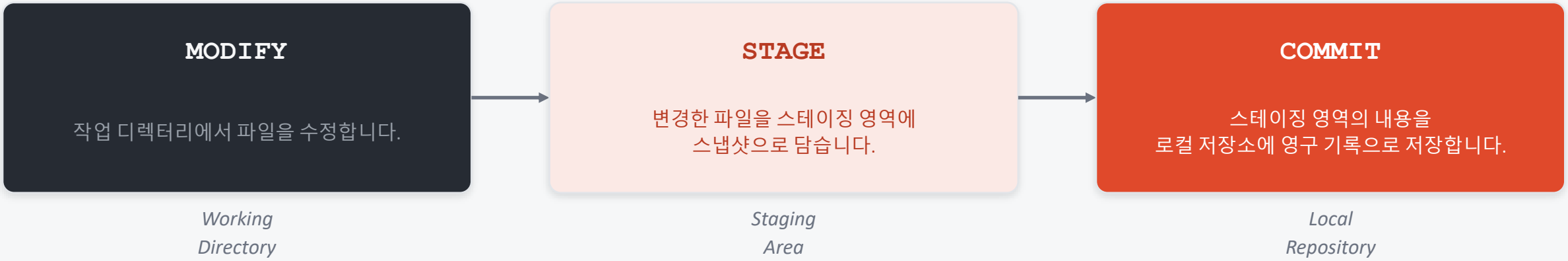
숨김 폴더 .git 확인

```
$ ls -a
.  ..  .git  Hello.java  README.md
```

핵심 .git이라는 숨김 디렉터리가 생성됩니다 — 이 폴더 안에 모든 커밋 이력과 설정이 저장됩니다. 이 폴더를 지우면 Git 이력이 전부 사라집니다.

4.3

기본 작업 순환: Edit → Stage → Commit

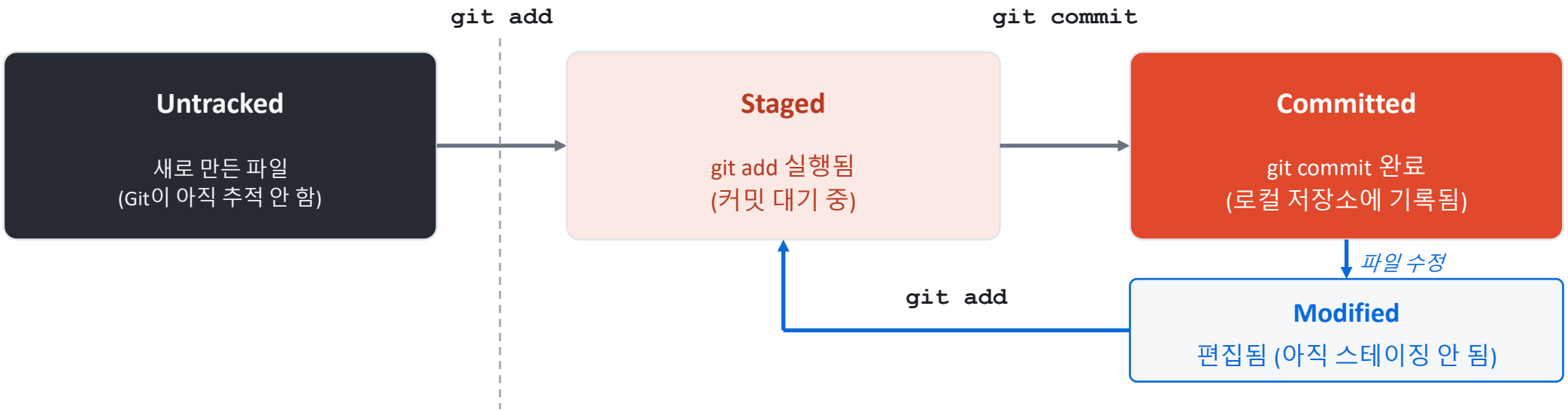



```
$ vi Hello.java
# 1. MODIFY – 파일 내용 수정
$ git add Hello.java
# 2. STAGE – 변경사항을 스테이징 영역에 담기
$ git commit -m "Create Hello Project"
# 3. COMMIT – 로컬 저장소에 영구 기록
```

4.4

파일 상태 생명주기

작업 트리의 파일은 untracked 또는 tracked 상태이며, tracked는 다시 Modified · Staged · Committed 세 상태로 나뉩니다.



명령어	역할
git status	untracked / staged / modified 등 현재 상태를 확인
git add <file>	파일 변경사항을 스테이징 영역에 추가 (untracked·modified → staged)
git commit	스테이징 영역의 모든 변경사항을 로컬 저장소에 커밋 (staged → committed)

4.5

git add — 변경사항 스테이징하기

먼저 git status로 현재 상태를 확인한 뒤, git add로 추적을 시작합니다.

git add

```
$ git status
Untracked files:
  Hello.java
  README.md

$ git add Hello.java
# 파일 하나만 스테이징
$ git add .
# 현재 디렉터리(하위 포함) 전체를 스테이징
$ git add *.java
# 와일드카드로 특정 확장자만 스테이징
```

되돌리기(Staging 취소) 실수로 스테이징했다면 git restore --staged <file> 명령어로 스테이징만 취소할 수 있습니다(작업 디렉터리의 작성/수정 내용은 삭제되지 않고 그대로 유지)

참고 git add 이후에도 아직 커밋 전이라 "향후 커밋할 변경 사항(Changes to be committed)"으로만 표시됩니다.

4.6

git commit — 커밋하기

스테이징 영역에 담긴 모든 변경사항을 하나의 커밋으로 저장합니다.

git commit

```
$ git commit -m "Initial commit: add Hello.java and README"
[main (root-commit) a1b2c3d] Initial commit: add Hello.java and README
 2 files changed, 10 insertions(+)
```

-m 옵션 -m 뒤에 커밋 메시지를 바로 적습니다. -m을 생략하면 vi 같은 편집기가 열려 메시지를 따로 작성해야 합니다.

커밋 메시지 "fix bug"처럼 모호하게 쓰지 말고, "무엇을 왜 바꿨는지"를 짧고 명확하게 적는 습관을 들이면 나중에 git log를 볼 때 큰 도움이 됩니다.

나쁜 예 `"fix bug"`

좋은 예 `"fix: 로그인 실패 시 에러 메시지가 표시되지 않던 문제 수정"`

4.7

git log — 커밋 이력 보기

각 커밋은 고유한 해시값(SHA-1 사용 시 40자리, SHA-256 사용 시 64자리)으로 식별되며, 식별 시에는 보통 앞 7자리만 단축형으로 사용합니다.

git log

```
$ git log
commit a1b2c3d4e5f6... (HEAD -> main)
Author: Hong Gildong <gildong@example.com>
Date:   Mon Sep 8 10:00:00 2025

    Initial commit: add Hello.java and README
```

한 줄씩 요약해서 보기

```
$ git log --oneline
a1b2c3d Initial commit: add Hello.java and README
```

최신 팁 git log --oneline --graph --all 을 쓰면 여러 브랜치의 이력을 그래프로 함께 볼 수 있습니다.

4.8

git status · git diff — 변경 확인하기

git status

```
$ git status
On branch main
Changes not staged for commit:
  modified:   Hello.java

# unstaged / staged / committed 상태를
# 한눈에 보여줌
```

git diff

```
$ git diff
-   System.out.println("Hello!");
+   System.out.println("Hi!");
```

언제 쓰나 커밋하기 전, "지금 무엇이 바뀌었는지"를 확인하는 습관을 들이면 실수를 크게 줄일 수 있습니다.
git status는 어떤 파일이 바뀌었는지, git diff는 그 파일에서 정확히 어떤 줄이 바뀌었는지(- 삭제된 줄, + 추가된 줄)를 보여줍니다.

4.8

git diff — 변경 확인하기



비교 대상 git diff는 무엇과 무엇을 비교하는지에 따라 결과가 달라집니다.
git diff(기본)는 작업 디렉터리 ↔ 스테이징 영역을,
git diff --cached는 스테이징 영역 ↔ HEAD를,
git diff HEAD는 작업 디렉터리 ↔ HEAD(마지막 커밋) 전체를 비교합니다.

5.1

.gitignore — 특정 파일 무시하기

빌드 산출물처럼 추적할 필요 없는 파일을 Git이 아예 무시하도록 지정합니다.

.gitignore

```
# .gitignore

# Java 클래스 파일
*.class

# 실행 파일
*.exe

# 정규식도 사용 가능: [oa]는 o 또는 a와 매치
*.[oa]

# temp 디렉터리 전체 (끝에 / 표시)
temp/
```

핵심 프로젝트 루트에 .gitignore 파일을 만들어 목록을 적으면, 이미 무시 대상인 파일은 git status의 "Untracked files" 목록에서 사라집니다.

5.2

.gitignore 쉽게 만들기

언어·프레임워크별 표준 .gitignore를 직접 손으로 안 써도 됩니다.

gitignore.io

언어/도구 이름을 입력하면
표준 .gitignore 내용을 즉시 생성해주는 웹 서비스입니다.

https://www.toptal.com/developers/gitignore

GitHub 공식 템플릿

GitHub이 직접 관리하는 언어별 .gitignore 모음으로,
새 저장소 생성 시 바로 선택할 수 있습니다.

https://github.com/github/gitignore

TIP IntelliJ, VS Code 등 대부분의 IDE도 프로젝트 생성 시 .gitignore를 자동으로 만들어줍니다.

6

실습: 나만의 첫 Git 저장소 만들기

New → Edit/Stage/Commit 순서로 직접 진행합니다.

사전 준비

```
# 1. Git 설치 확인
$ sudo dnf install -y git
$ git --version

# 2. 실습용 Java 환경 준비
$ sudo dnf install -y java-17-amazon-corretto-devel
```

단계	실습 내용
설정	git config로 사용자 정보 및 기본 브랜치(main) 설정
시작	git init으로 hello-git 저장소 생성
작업 순환	Hello.java 작성 → git add → git commit 반복
확인	git status, git log --oneline 으로 상태·이력 점검
무시 설정	.gitignore로 *.class 등 빌드 산출물 제외

정리하기

1. VCS는 코드 변경 이력을 기록해 백업·되돌리기·협업을 가능하게 합니다. Git은 분산형 VCS로, 매 커밋마다 스냅샷을 저장합니다.
2. git config로 사용자 정보를, git init으로 새 저장소를 만듭니다.
3. 파일은 Working Directory → (git add) → Staging Area → (git commit) → Local Repository 순서로 이동합니다.
4. git status/git log/git diff로 현재 상태와 변경 내역, 이력을 확인합니다.
5. .gitignore로 추적할 필요 없는 파일을 제외합니다.
6. 다음 시간(Git RemoteRepo)에서는 GitHub과 연결해 push/pull 다룹니다.